

Exercise 4: DATA PREPROCESSING – HANDLING MISSING VALUES

Write a python program to impute missing values with various techniques on given dataset.

1. Remove rows/ attributes
2. Replace with mean or mode
3. Write a python program to perform transformation of data using Discretization (Binning) and normalization (MinMaxScaler or MaxAbsScaler) on given dataset.

1. Remove rows/ attributes

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv("D:/diabetes.csv")
# filling missing value using fillna()
print(df.fillna(0))
# filling a missing value with previous value
print(df.fillna(method = 'pad'))
#Filling null value with the next ones
print(df.fillna(method = 'bfill'))
# filling a null values using fillna()
print(df["gender"].fillna("No Gender", inplace = True))
# will replace Nan value in dataframe with value -99
print(df.replace(to_replace = np.nan, value = -99))
# using dropna() function to remove rows having one Nan
print(df.dropna())
# using dropna() function to remove rows with all Nan
print(df.dropna(how = 'all'))
# using dropna() function to remove column having one Nan
print(df.dropna(axis = 1))
```



2. Replace with mean or mode

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv("D:/diabetes.csv")
# filling missing value using fillna()
print(df.fillna(0))
# filling a missing value with previous value
print(df.fillna(method='pad'))
#Filling null value with the next ones
print(df.fillna(method='bfill'))
# filling a null values using fillna()
print(df["gender"].fillna("No Gender", inplace = True))
# will replace Nan value in dataframe with value -99
print(df.replace(to_replace = np.nan, value = -99))
# using dropna() function to remove rows having one Nan
print(df.dropna())
# using dropna() function to remove rows with all Nan
print(df.dropna(how = 'all'))
# using dropna() function to remove column having one Nan
print(df.dropna(axis = 1))
print(df["Age"].fillna(df["Age"].mean()))
```

3. Perform transformation of data using Discretization (Binning)

Binning can also be used as a discretization technique. Discretization refers to the process of converting or partitioning continuous attributes, features or variables to discretized or nominal attributes/ features/ variables/ intervals.

For example, attribute values can be discretized by applying equal-width or equal-frequency binning, and then replacing each bin value by the bin mean or median, as in smoothing by bin means or smoothing by bin medians, respectively. Then the continuous values can be converted to a nominal or discretized value which is same as the value of their corresponding bin.

There are basically two types of binning approaches –

Equal width (or distance) binning: The simplest binning approach is to partition the range of the variable into k equal-width intervals. The interval width is simply the range $[A, B]$ of the variable divided by k , $w = (B-A) / k$

Thus, i^{th} interval range will be $[A + (i-1)w, A + iw]$ where $i = 1, 2, 3, \dots, k$

Skewed data cannot be handled well by this method.

Equal depth (or frequency) binning : In equal-frequency binning we divide the range $[A, B]$ of the variable into intervals that contain (approximately) equal number of points; equal frequency may not be possible due to repeated values.

There are three approaches to perform smoothing –

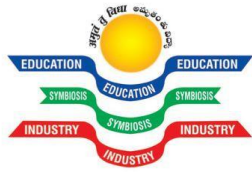
Smoothing by bin means : In smoothing by bin means, each value in a bin is replaced by the mean value of the bin.

Smoothing by bin median : In this method each bin value is replaced by its bin median value.

Smoothing by bin boundary : In smoothing by bin boundaries, the minimum and maximum values in a given bin are identified as the bin boundaries. Each bin value is then replaced by the closest boundary value.

Example:

Sorted data for price(in dollar) : 2, 6, 7, 9, 13, 20, 21, 25, 30



Partition using equal frequency approach:

Bin 1 : 2, 6, 7
Bin 2 : 9, 13, 20
Bin 3 : 21, 24, 30

Smoothing by bin mean :

Bin 1 : 5, 5, 5
Bin 2 : 14, 14, 14
Bin 3 : 25, 25, 25

Smoothing by bin median :

Bin 1 : 6, 6, 6
Bin 2 : 13, 13, 13
Bin 3 : 24, 24, 24

Smoothing by bin boundary :

Bin 1 : 2, 7, 7
Bin 2 : 9, 9, 20
Bin 3 : 21, 21, 30

Smoothing by bin means:

```
import numpy as np

from sklearn.linear_model import LinearRegression

from sklearn import linear_model

# import statsmodels.api as sm

import statistics

import math

from collections import OrderedDict

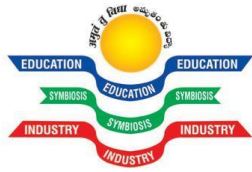
x = []

print("enter the data:")

x = list(map(float, input().split()))

print("enter the number of bins:")

bi = int(input())
```



X_dict will store the data in sorted order

```
X_dict = OrderedDict()
```

x_old will store the original data

```
x_old = {}
```

x_new will store the data after binning

```
x_new = {}
```

```
for i in range(len(x)):
```

```
    X_dict[i]= x[i]
```

```
    x_old[i]= x[i]
```

```
x_dict = sorted(X_dict.items(), key = lambda x: x[1])
```

list of lists(bins)

```
binn = []
```

a variable to find the mean of each bin

```
avrg = 0
```

```
i = 0
```

```
k = 0
```

```
num_of_data_in_each_bin = int(math.ceil(len(x)/bi))
```

performing binning

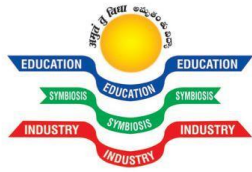
```
for g, h in X_dict.items():
```

```
    if(i<num_of_data_in_each_bin):
```

```
        avrg = avrg + h
```

```
        i = i + 1
```

```
    elif(i == num_of_data_in_each_bin):
```



```
k = k + 1

i = 0

binn.append(round(avrg / num_of_data_in_each_bin, 3))

avrg = 0

avrg = avrg + h

i = i + 1

rem = len(x)% bi

if(rem == 0):

    binn.append(round(avrg / num_of_data_in_each_bin, 3))

else:

    binn.append(round(avrg / rem, 3))

# store the new value of each data

i = 0

j = 0

for g, h in X_dict.items():

    if(i<num_of_data_in_each_bin):

        x_new[g]= binn[j]

        i = i + 1

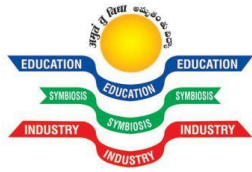
    else:

        i = 0

        j = j + 1

        x_new[g]= binn[j]

        i = i + 1
```



```
print("number of data in each bin")

print(math.ceil(len(x)/bi))

for i in range(0, len(x)):

    print('index {2} old value {0} new value {1}'.format(x_old[i], x_new[i], i))
```

Smoothing by bin medians:

```
import numpy as np

from sklearn.linear_model import LinearRegression

from sklearn import linear_model

# import statsmodels.api as sm

import statistics

import math

from collections import OrderedDict

x = []

print("enter the data")

x = list(map(float, input().split()))

print("enter the number of bins")

bi = int(input())

# X_dict will store the data in sorted order

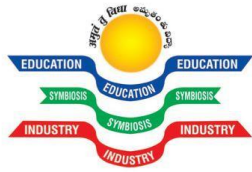
X_dict = OrderedDict()

# x_old will store the original data

x_old = {}

# x_new will store the data after binning

x_new = {}
```



```
for i in range(len(x)):
    X_dict[i]= x[i]
    x_old[i]= x[i]
x_dict = sorted(X_dict.items(), key = lambda x: x[1])

# list of lists(bins)

binn =[]

# a variable to find the mean of each bin

avrg =[]

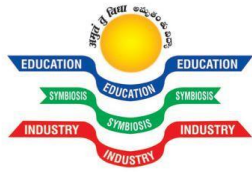
i = 0

k = 0

num_of_data_in_each_bin = int(math.ceil(len(x)/bi))

# performing binning

for g, h in X_dict.items():
    if(i<num_of_data_in_each_bin):
        avrg.append(h)
        i = i + 1
    elif(i == num_of_data_in_each_bin):
        k = k + 1
        i = 0
        binn.append(statistics.median(avrg))
        avrg =[]
        avrg.append(h)
        i = i + 1
```



```
binn.append(statistics.median(avrg))
```

```
# store the new value of each data
```

```
i = 0
```

```
j = 0
```

```
for g, h in X_dict.items():
```

```
    if(i<num_of_data_in_each_bin):
```

```
        x_new[g]= round(binn[j], 3)
```

```
        i = i + 1
```

```
    else:
```

```
        i = 0
```

```
        j = j + 1
```

```
        x_new[g]= round(binn[j], 3)
```

```
        i = i + 1
```

```
print("number of data in each bin")
```

```
print(math.ceil(len(x)/bi))
```

```
for i in range(0, len(x)):
```

```
    print('index {2} old value {0} new value {1}'.format(x_old[i], x_new[i], i))
```

Smoothing by bin boundaries:

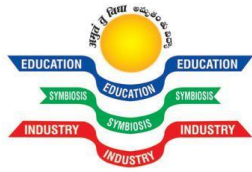
```
import numpy as np
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn import linear_model
```

```
# import statsmodels.api as sm
```

```
import statistics
```



```
import math

from collections import OrderedDict

x =[]

print("enter the data")

x = list(map(float, input().split()))

print("enter the number of bins")

bi = int(input())

# X_dict will store the data in sorted order

X_dict = OrderedDict()

# x_old will store the original data

x_old ={}

# x_new will store the data after binning

x_new ={}

for i in range(len(x)):

    X_dict[i]= x[i]

    x_old[i]= x[i]

x_dict = sorted(X_dict.items(), key = lambda x: x[1])

# list of lists(bins)

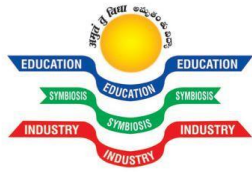
binn =[]

# a variable to find the mean of each bin

avrg =[]

i = 0

k = 0
```



```
num_of_data_in_each_bin = int(math.ceil(len(x)/bi))
```

```
for g, h in X_dict.items():
```

```
    if(i<num_of_data_in_each_bin):
```

```
        avrg.append(h)
```

```
        i = i + 1
```

```
    elif(i == num_of_data_in_each_bin):
```

```
        k = k + 1
```

```
        i = 0
```

```
        binn.append([min(avrg), max(avrg)])
```

```
        avrg = []
```

```
        avrg.append(h)
```

```
        i = i + 1
```

```
binn.append([min(avrg), max(avrg)])
```

```
i = 0
```

```
j = 0
```

```
for g, h in X_dict.items():
```

```
    if(i<num_of_data_in_each_bin):
```

```
        if(abs(h-binn[j][0]) >= abs(h-binn[j][1])):
```

```
            x_new[g]= binn[j][1]
```

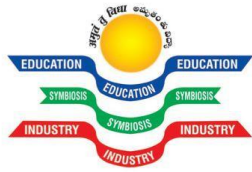
```
            i = i + 1
```

```
        else:
```

```
            x_new[g]= binn[j][0]
```

```
            i = i + 1
```

```
    else:
```



```
i = 0
j = j + 1
if(abs(h-binn[j][0]) >= abs(h-binn[j][1])):
    x_new[g]= binn[j][1]
else:
    x_new[g]= binn[j][0]
i = i + 1

print("number of data in each bin")

print(math.ceil(len(x)/bi))

for i in range(0, len(x)):

    print('index {2} old value {0} new value {1}'.format(x_old[i], x_new[i], i))
```

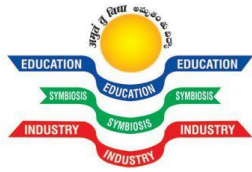
3. Perform transformation of data using normalization (MinMaxScaler or MaxAbsScaler) on given dataset. In preprocessing, standardization of data is one of the transformation task. Standardization is scaling features to lie between a given minimum and maximum value, often between zero and one, or so that the maximum absolute value of each feature is scaled to unit size. This can be achieved using MinMaxScaler or MaxAbsScaler, respectively.

The motivation to use this scaling include robustness to very small standard deviations of features and preserving zero entries in sparse data.

example of a normalization

```
from numpy import asarray
from sklearn.preprocessing import MinMaxScaler

# define data
data = asarray([[100, 0.001],[8, 0.05],[50, 0.005],[88, 0.07],[4, 0.1]])
print(data)
```



```
# define min max scaler
```

```
scaler = MinMaxScaler()
```

```
# transform data
```

```
scaled = scaler.fit_transform(data)
```

```
print(scaled)
```